Maryam Abrahams - ABRMAR043
Saiyantha Moodley - MDLSAI010
Daahil Lakha - LKHDAA001

Introduction

The aim of this assignment was to develop a networking chat application in Python that allows for multiple users to concurrently communicate over the network with each other. This report will go into depth about the workings of our application, including a description of our applications features and how networking elements, threading and database structures have been used in its implementation and design.

Architecture Overview

Our chat application design uses both the client-server and peer-to-peer network architecture to perform different functions. This was intended to take advantage of the benefits of each architecture type and maintain the flow of messages through our application without disruptions.

The client-server architecture is used for most of the applications functionality such as authentication, group and private messaging and the retrieval of message history. The server also handles the majority of database interactions.

The peer-to-peer architecture is used only for client file-sharing capabilities. This is to ensure users can safely transfer files without burdening the server with large files that can create congestion and potentially disrupt other application features.

Both client-server and peer-to-peer interactions are executed using TCP connections. This was intended for the purpose of secure , reliable data transfer.

The main components of the system consists of:

- A server
- A client application
- A database

The client application allows for user interaction, the server is responsible for database management and user request processing while the database is used for storage of user, message and chat information.

Roles of Server and Client

The server runs continuously and listens for incoming user connections. Once a new client connects to a server it receives its own thread for communication. This allows multiple users to connect and communicate via the server without delays.

Clients connect to the server using a TCP connection and then input commands into the terminal from the specified command list. Users can perform authentication, private messaging as well as creating and messaging on groups through terminal commands.

The server is responsible for the interpretation of the clients requests and for ensuring these requests are fulfilled. The server coordinates the clients messages and ensures they are correctly received by the intended recipients. The server also manages most of the database access and is responsible for authenticating users before they can access the chat applications features.

Peer-to-Peer Communication

For the file sharing clients connect directly to each other in a peer-to-peer setup using a separate dedicated port and can then send files directly to each other. Each individual client runs its own personal server used solely for receiving file transfers from other users. This was aimed to reduce congestion of data flow through the server. The receiver's port number is retrieved from the database using the intended recipient's username. This is possible as all users are assigned a port for sending files when they join the server and every user's username is unique.

When a user wishes to send a file they must input the correct command into the terminal in the correct syntax. The client recognises the command as a file transfer and does not send the request to the main server. The request is processed within the client application. The client application creates a TCP socket connection with the recipient client's file server and then once the connection is accepted the file to be transferred is sent in packets. On the receiver's end the file is received and written to a folder on the client’s computer for all file transfers. The file receiver method also runs in’s own thread separate from the main thread which handles incoming messages. This allows the user to maintain communication capabilities while processing the file transfer and also allows for multiple files to be sent to the same user at the same time.

TCP and UDP Connections

TCP connections were used for almost all of our chat application features. The client and server have a TCP connection which is used for the sending of all requests from client to server. This is used for processes such as authentication, messaging and group chat management. TCP was used as it is the more reliable choice for the connection. These features of the chat app required order and guaranteed delivery of packets which is something that UDP cannot offer. guaranteeing atomicity in our database and correctly ordered communication.

We decided to only use UDP for sending and receiving small online status messages between clients. When a user logs into the system, the client sends a UDP message to the server indicating that the user is online. The server keeps a small array containing all the current online clients. The server then uses this list to broadcast this message to other connected clients so they can display the user's status. Similarly, when a user disconnects or logs out, an offline notification is broadcast to the other clients. UDP was chosen for this purpose because it is faster than TCP and status is not necessarily a critical function therefore the reliability of UDP for this function is not a deep concern.

Database Implementation

We have made use of an SQLite database to store user and chat information. The server is used to update and retrieve information from the database. The database ensures users information is retained when they logout of the chat application.

The database has 4 tables:

1. users
    - This table contains the users username and password and is used by the server for user authentication (login/sign-up). It also contains the file sharing port number that is used to create peer-to-peer connections between clients.
2. chats
    - The chats table stores a chat id and what type of chat it is (group or private)
3. chat_members
    - This table stores the chat ID and the user ID’s of all the members of the chat
4. messages
    - This table has a message ID, chat ID and sender ID and message content is used to queue messages before they are sent.

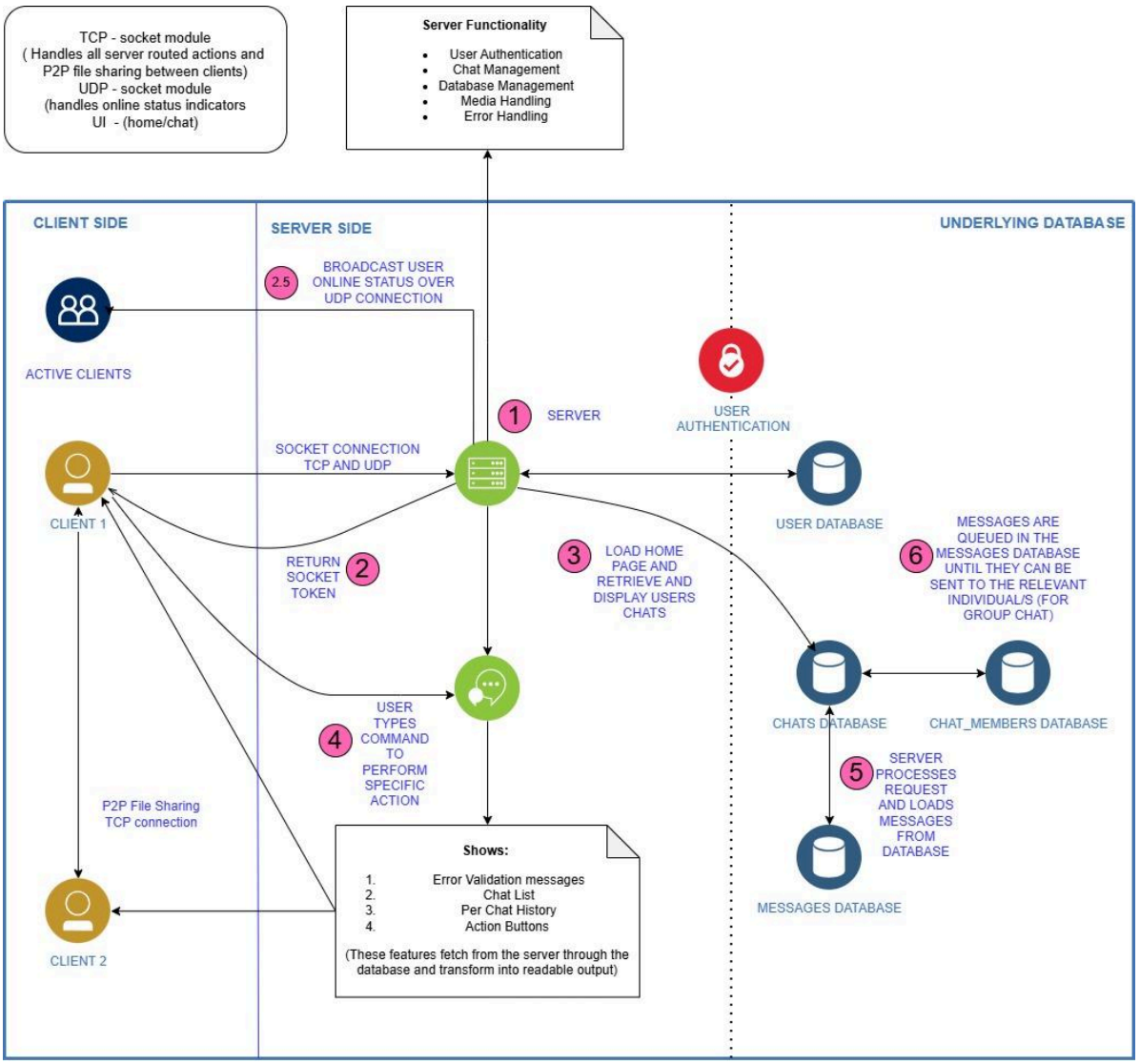


Diagram of System Architecture showing the system components and the different connections and network architectures used in our application.

System Functions

The chat application performs the required functions of allowing users to communicate over the network either privately or in a group setting as well as a global chat feature which allows a user to communicate with all online users. It also allows users to share multimedia files. Additionally, our application has user authentication and a recent chat history feature. The following explains the procedure of each of the above features:

1. **User authentication** - ensures each user only has strict access to their own chats
    a. The client is prompted to either login or sign-up
    b. If the client is new they must enter a new username and password before they can access the chat app
    c. If the client is existing they must login with their previously determined username and password
    d. The input is sent to the server using the TCP socket connection
    e. The server makes use of the database to verify the users input
    f. If the input is valid the user is authenticated and can proceed to use the other chat features

2. **Private Message** - used for a client to message another **single** registered chat app user
    a. The user enters their intended message using the correct provided message syntax /pm <username> <message>
    b. The message is sent to the server using the TCP socket connection
    c. The message is analysed to determine the correct recipient and separate the message (This is done through sql queries to the database checking user membership to chats as well as online status to know whether or not to be broadcasted)
    d. The server then sends the message to the correct recipient
    e. The message is displayed in the recipients console
    f. The message is saved to the user's history in our database and if a user is offline the message is displayed to their console in a history section upon their next login

3. **Group Message**
    a. Users can create and join groups using the appropriate syntax commands in their console
    b. The user enters their intended message using the correct provided message syntax /group <group name> <message>
    c. The message is sent to the server using the TCP socket connection
    d. The message is analysed to determine the correct group chat name and separate the message
    e. The server accesses the database to determine the members of that group chat using the chat_ID of the group chat
    f. The server then sends the message to all the members of the group
    g. The message is displayed in the recipients' consoles

4. **Global Message**
    a. When a user enters a message not specific to a command it automatically gets identified as a “global” message
    b. The message is sent to the server using the TCP socket connection
    c. Online users are identified by the server
    d. The server then broadcasts the message to all online users
    e. The message is displayed in the recipients console

5. **Message History**
    a. Any message sent by an online user to any authenticated user (user existing in our database) gets stored to our database into a user-specific chat history whether they are online or offline
    b. When a user that is not new logs back into the system on startup their chat-history is fetched from the database and displayed on their console screen. This includes all messages addressed to the user from any chat they are affiliated with (ie: global, pms and joined group chats)

6. **File Sharing**
    a. The user enters the command for file sharing in the correct syntax into their console /file <username> <filename> OR /file <group name> <filename>
    b. The client application detects that the command is for file sharing
    c. The client application parses the command to determine the correct recipient and the filename
    d. The intended recipients port number is retrieved from the database
    e. A TCP connection is created between the sender and the the recipient
    f. The file is transmitted over this connection
    g. The file is written to a folder in the receivers directory

7. **Online status indicator**
    a. The user either signs-up or logs in to the chat application
    b. The user makes a UDP connection with the server
    c. The server broadcasts to all other active clients via the UDP connection that the new user is online
    d. The message is received by the other users UDP ports
    e. The status update is displayed on the clients consoles
    f. Similarly, when a user logs out, the same procedure occurs except the message displayed indicates that the user is now offline.

Screenshots of Application

```
Connected to server at localhost:12001

-- Welcome to WhatsUp --
1. Login
2. Sign-up
q. Quit
Select an option by number or q to quit: 2
Enter a new username: Maryam
Enter a new password: 12345
Sign-up successful!

========================================
       RECENT CHAT HISTORY
========================================
  No recent activity.

========================================


Authenticated successfully!

-- Command Pallete --
Follow the syntax for each command to execute automatically:
1. Private Message (Syntax: /pm <user> <message>)
2. Message Group (Syntax: /group <group_id> <message>)
3. Send File (Syntax: /file <user> <filename> OR /file <group_name> <filename> )
4. Create New Group (Syntax: /create <group_name>)
5. Join Group (Syntax: /join <group_id>)
Type 'logout' to return to menu.
Type 'quit' to exit.
> File server running on port: 55237
logout
Logging out...
```

Figure 1.1 Shows the user Maryam signing-up to the chat application.

```
TCP Server is ready to receive connections on port 12001...
UDP Status Server ready on port 13000...
New connection from ('127.0.0.1', 57028)
[INFO] New client handler started for ('127.0.0.1', 57028)
New connection from ('127.0.0.1', 58648)
[INFO] New client handler started for ('127.0.0.1', 58648)
New connection from ('127.0.0.1', 45146)
[INFO] New client handler started for ('127.0.0.1', 45146)
[AUTH] User Maryam registered.
```

Figure 1.2 Shows server console during user sign-up

```
Connected to server at localhost:12001

-- Welcome to WhatsUp --
1. Login
2. Sign-up
q. Quit
Select an option by number or q to quit: 1
Enter username: Maryam
Enter password:  12345

========================================
       RECENT CHAT HISTORY
========================================

>>> Global Chat <<<
  (Empty)

>>> PM with Saiyantha <<<
  - <Hi Maryam!>
  - <Hi Saiyanini!!!!!! How are you?>
  - <Emotionally? meh Physically? meh Financially? terrible>
  - <How is the comp sci assignment going brooooo?>

========================================


Authenticated successfully!

-- Command Pallete --
Follow the syntax for each command to execute automatically:
1. Private Message (Syntax: /pm <user> <message>)
2. Message Group (Syntax: /group <group_id> <message>)
3. Send File (Syntax: /file <user> <filename> OR /file <group_name> <filename> )
4. Create New Group (Syntax: /create <group_name>)
5. Join Group (Syntax: /join <group_id>)
Type 'logout' to return to menu.
Type 'quit' to exit.
> File server running on port: 44217
```

Figure 1.3 The user then logs out and then selects the Login option and is able to be successfully authenticated using the previously chosen username and password.

```
TCP Server is ready to receive connections on port 12001...
UDP Status Server ready on port 13000...
New connection from ('127.0.0.1', 52198)
[INFO] New client handler started for ('127.0.0.1', 52198)
[AUTH] User Maryam authenticated.
```

Figure 1.4 Shows the server consoles response to a user successfully logging-in

```
-- Welcome to WhatsUp --
1. Login
2. Sign-up
q. Quit
Select an option by number or q to quit: 1
Enter username: Maryam
Enter password:  apples
Login failed! (Server said: Invalid username or password.)
```

Figure 1.5 Shows the user incorrectly logging-in

```
[STATUS] Maryam is online
/pm <Maryam> <Hi Miss Mary Berry>
> [PM to Maryam]: <Hi Miss Mary Berry>
/pm <Maryam> <I liked your dress today>
> [PM to Maryam]: <I liked your dress today>

[RECEIVED] PM from Maryam: <Hi Saiyanini!!!!!!>

[RECEIVED] PM from Maryam: <Thank you!>
/pm <Maryam> <How are you doll?>
> [PM to Maryam]: <How are you doll?>

[RECEIVED] PM from Maryam: <I'm suffering saiyanini>
/pm <Maryam> <Why Mary>
> [PM to Maryam]: <Why Mary>

[RECEIVED] PM from Maryam: <Bro this assignment bro. It's killing me.>
```

Saiyantha's Console

```
[RECEIVED] PM from Saiyantha: <Hi Miss Mary Berry>

[RECEIVED] PM from Saiyantha: <I liked your dress today>
/pm <Saiyantha> <Hi Saiyanini!!!!!!>
> [PM to Saiyantha]: <Hi Saiyanini!!!!!!>
/pm <Saiyantha> <Thank you!>
> [PM to Saiyantha]: <Thank you!>

[RECEIVED] PM from Saiyantha: <How are you doll?>
/pm <Saiyantha> <I'm suffering saiyanini>
> [PM to Saiyantha]: <I'm suffering saiyanini>

[RECEIVED] PM from Saiyantha: <Why Mary>
/pm <Saiyantha> <Bro this assignment bro. It's killing me.>
> [PM to Saiyantha]: <Bro this assignment bro. It's killing me.>
```

Maryam's Console

Figure 2.1 Shows a private message between 2 clients

```
[CMD] Saiyantha sent command: /pm
[BROADCAST] Target: ['Maryam'], Delivered to: 1 users.
[CMD] Saiyantha sent command: /pm
[BROADCAST] Target: ['Maryam'], Delivered to: 1 users.
[CMD] Maryam sent command: /pm
[BROADCAST] Target: ['Saiyantha'], Delivered to: 1 users.
[CMD] Maryam sent command: /pm
[BROADCAST] Target: ['Saiyantha'], Delivered to: 1 users.
[CMD] Saiyantha sent command: /pm
[BROADCAST] Target: ['Maryam'], Delivered to: 1 users.
[CMD] Maryam sent command: /pm
[BROADCAST] Target: ['Saiyantha'], Delivered to: 1 users.
[CMD] Saiyantha sent command: /pm
[BROADCAST] Target: ['Maryam'], Delivered to: 1 users.
[CMD] Maryam sent command: /pm
[BROADCAST] Target: ['Saiyantha'], Delivered to: 1 users.
```

Figure 2.2 Shows the console of the server while the clients are messaging

```
[STATUS] Daahil is online
/create <csc project>
> CONFIRM: Group 'csc project' created successfully!
/group <csc project> <Dale can you come help me with the report>
>
[RECEIVED] Group csc project from Saiyantha: <Dale can you come help me with the report>

[RECEIVED] Group csc project from Daahil: <Can't>

[RECEIVED] Group csc project from Daahil: <I'm getting a chocolate croissant now>

[RECEIVED] Group csc project from Daahil: <I'll come help later>

[RECEIVED] Group csc project from Maryam: <This guy!>
```

Saiyantha's console

```
/join <csc project>
> CONFIRM: Joined group 'csc project'.

[RECEIVED] Group csc project from Saiyantha: <Dale can you come help me with the report>
/group <csc project> <Can't>
>
[RECEIVED] Group csc project from Daahil: <Can't>
/group <csc project> <I'm getting a chocolate croissant now>
>
[RECEIVED] Group csc project from Daahil: <I'm getting a chocolate croissant now>
/group <csc project> <I'll come help later>
>
[RECEIVED] Group csc project from Daahil: <I'll come help later>

[RECEIVED] Group csc project from Maryam: <This guy!>
```

Daahil's console

Figure 3.1 Shows user Saiyantha creating a group chat called "csc project". The users Daahil and Maryam join the group chat using the join command. The messages sent are then broadcast to all 3 members of the group chat.

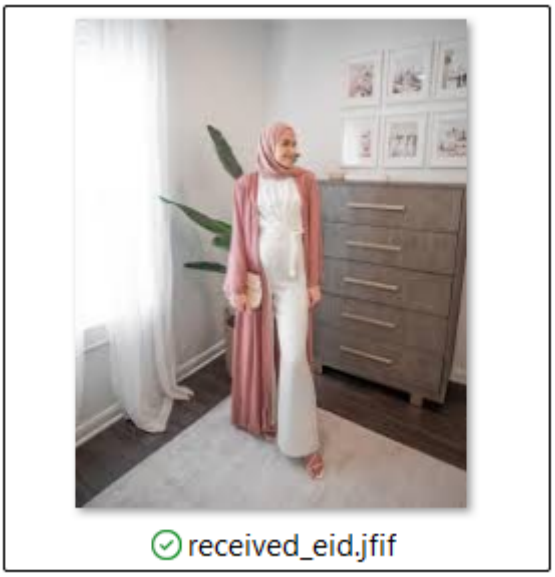
```
[CMD] Saiyantha sent command: /create
[CMD] Maryam sent command: /join
[CMD] Daahil sent command: /join
[CMD] Daahil sent command: /join
[CMD] Saiyantha sent command: /group
[BROADCAST] Target: ['Maryam', 'Saiyantha', 'Daahil'], Delivered to: 3 users.
[CMD] Daahil sent command: /group
[BROADCAST] Target: ['Maryam', 'Saiyantha', 'Daahil'], Delivered to: 3 users.
[CMD] Daahil sent command: /group
[BROADCAST] Target: ['Maryam', 'Saiyantha', 'Daahil'], Delivered to: 3 users.
[CMD] Daahil sent command: /group
[BROADCAST] Target: ['Maryam', 'Saiyantha', 'Daahil'], Delivered to: 3 users.
[CMD] Maryam sent command: /group
[BROADCAST] Target: ['Maryam', 'Saiyantha', 'Daahil'], Delivered to: 3 users.
```

Figure 3.2 Shows the console during group chat messaging

```
[RECEIVED] PM from Saiyantha: <Hey doll what are you wearing for eid>
/file <Saiyantha> <eid.jfif>
File sent successfully.
```

```
/pm <Maryam> <Hey doll what are you wearing for eid>
> [PM to Maryam]: <Hey doll what are you wearing for eid>

Received file: eid.jfif, saved as received_files/received_eid.jfif
```

Figure 4.1 Shows the file sharing feature of our application



"C:\Users\saiya\OneDrive - University of Cape Town\CSC3002F\CSC3002F-Networks-Assignment-Socket-Programming-Project\client\received_files\received_eid.jfif"

Figure 4.2 above shows the file that was sent between the 2 clients and the file path of the received file

```
========================================
       RECENT CHAT HISTORY
========================================

>>> Global Chat <<<
  (Empty)

>>> PM with Maryam <<<
  - <Hi Maryam!>
  - <Hi Saiyanini!!!!!! How are you?>
  - <Emotionally? meh Physically? meh Financially? terrible>
  - <How is the comp sci assignment going brooooo?>
  - <Hi Miss Mary Berry!>
  - <I liked your dress today>
  - <Hi Miss Mary Berry>
  - <I liked your dress today>
  - <Hi Saiyanini!!!!!!>
  - <Thank you!>
  - <How are you doll?>
  - <I'm suffering saiyanini>
  - <Why Mary>
  - <Bro this assignment bro. It's killing me.>
  - <Hey doll what are you wearing for eid>

>>> Group 4 <<<
  - <Dale can you come help me with the report>
  - <Can't>
  - <I'm getting a chocolate croissant now>
  - <I'll come help later>
  - <This guy!>

========================================
```

Figure 5 shows the recent chat history that appears when a user logs in.

```
[STATUS] Saiyantha is online

[RECEIVED] Group csc project from Saiyantha: <Hi Maryam, Hi Daahil. I must just go eat quickly and then I'll come join the call.>

[RECEIVED] PM from Saiyantha: <Didn't Daahil look nice today.>

[STATUS] Saiyantha is offline
```

Figure 6 shows the UDP status indicators that show when a user is logged-in or not.

Wireshark Captures

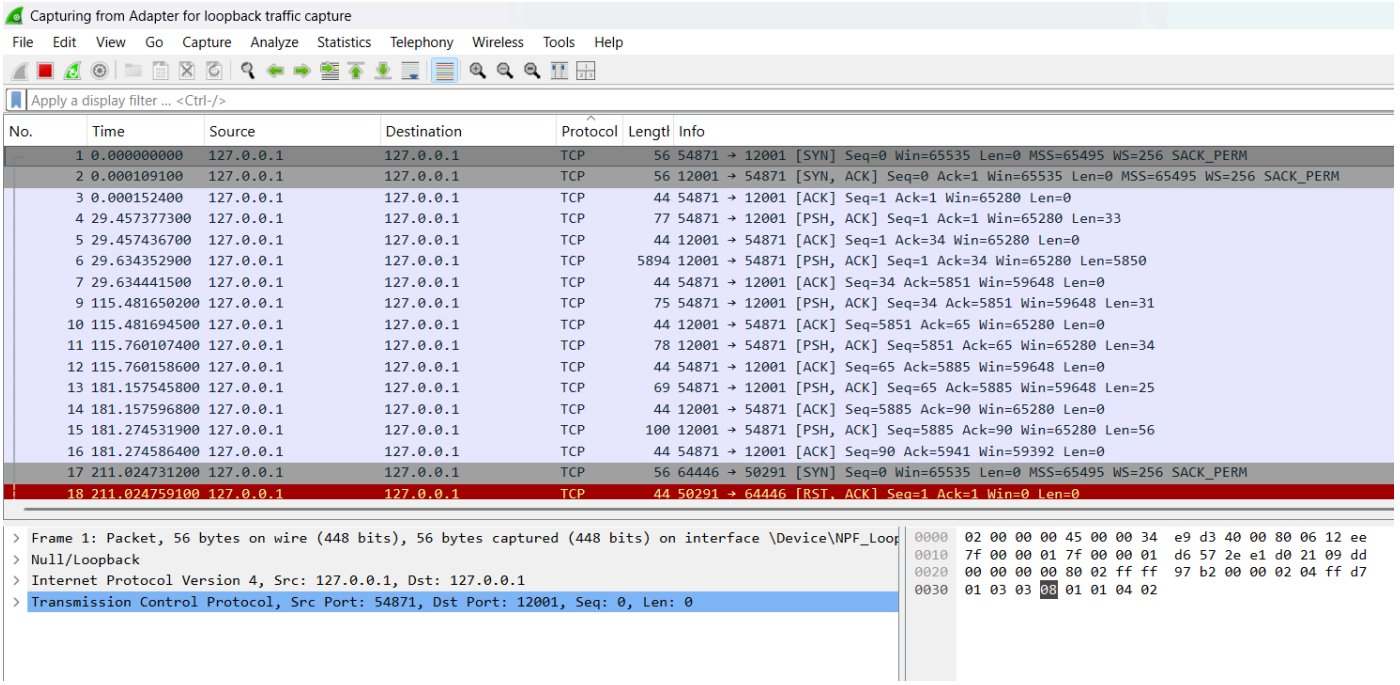


Figure 7.1 shows a screenshot of all the packets captured by wireshark of our network application.

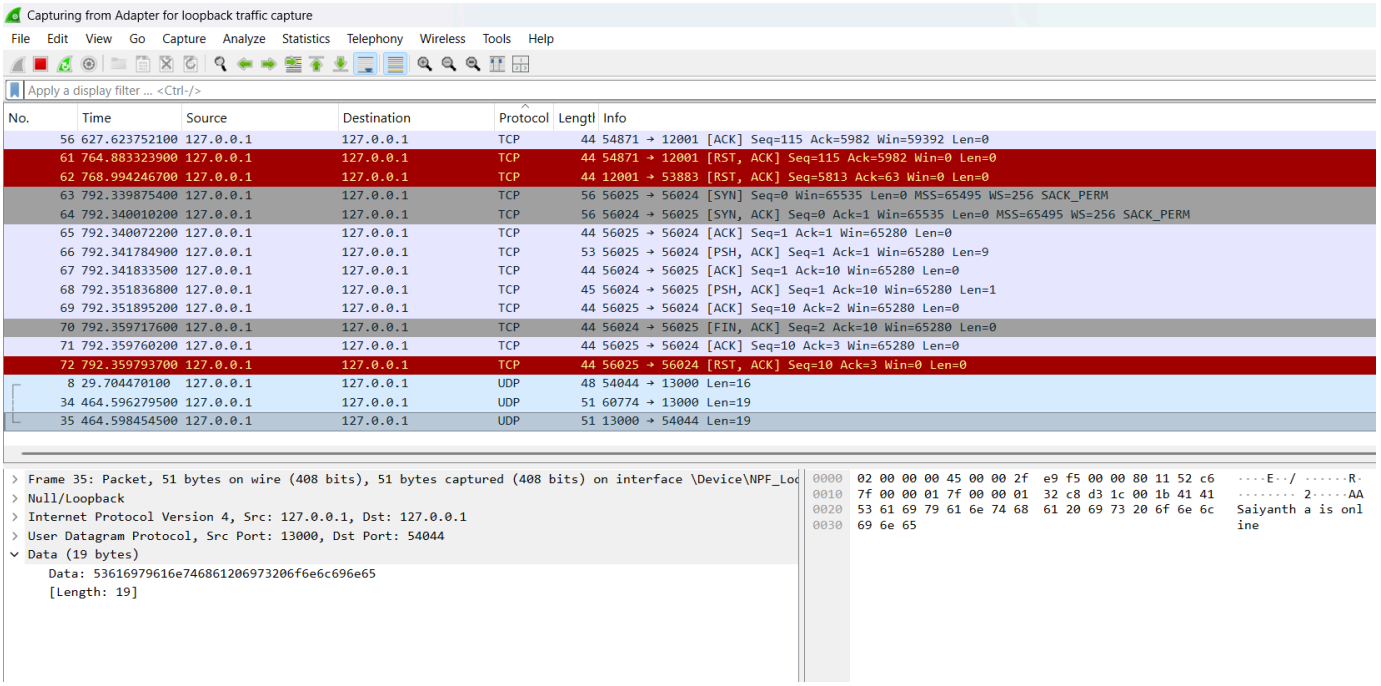


Figure 7.2 A continuation of the above

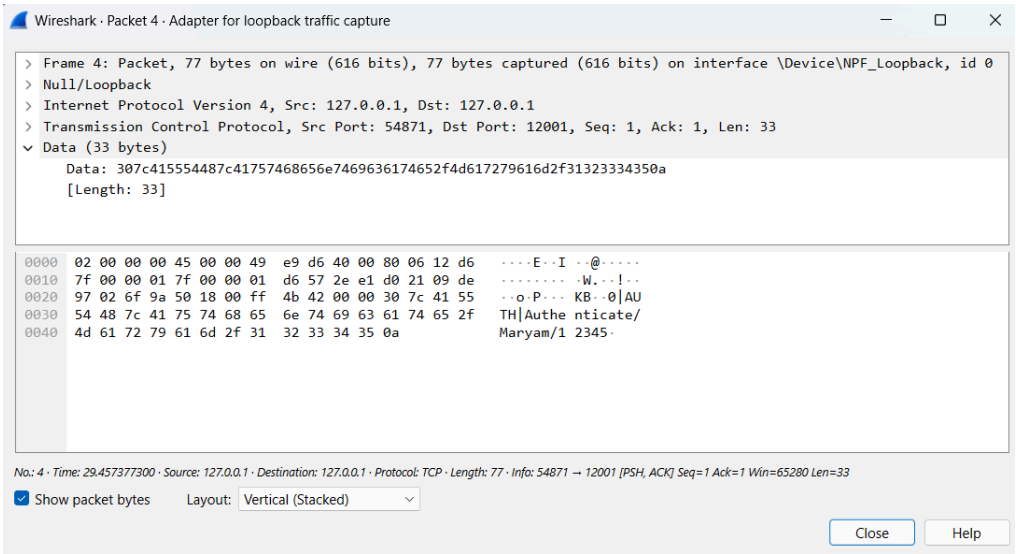


Figure 8.1 A capture of the packet sent from client to server during authentication.

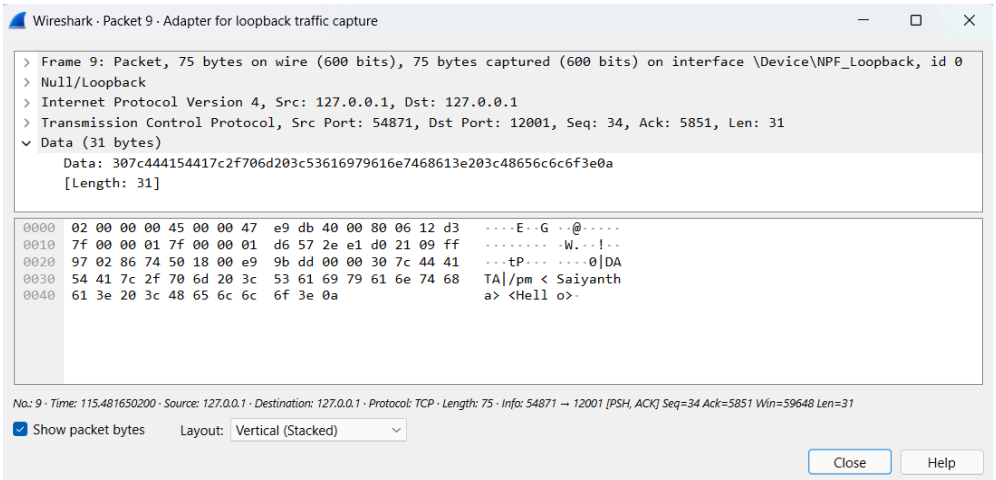


Figure 8.2 A capture of a packet that contains a private message

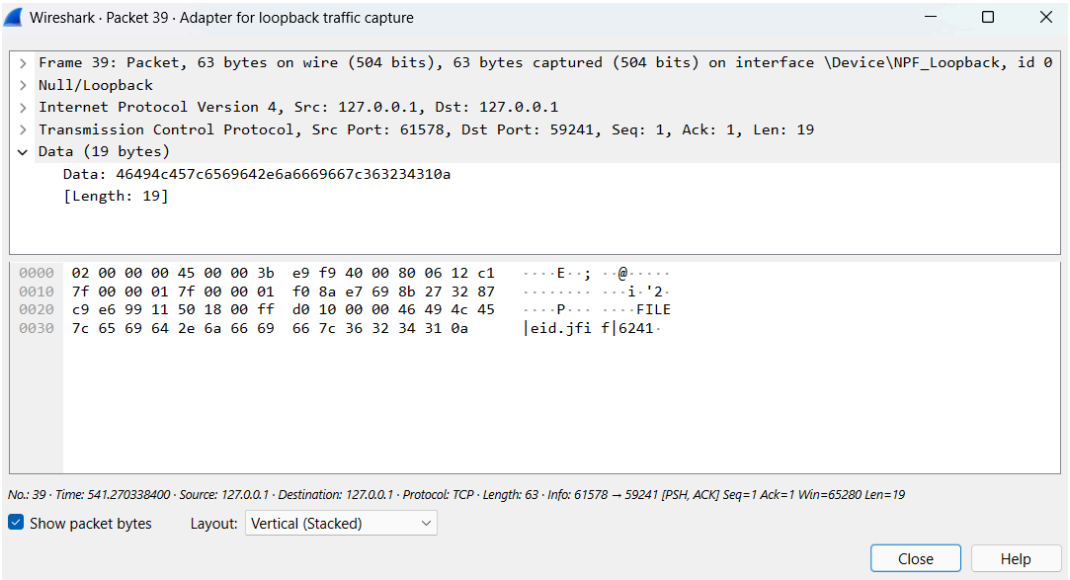


Figure 8.3 A capture of a packet containing the header that is sent before a file is shared. It shows the file name and size.

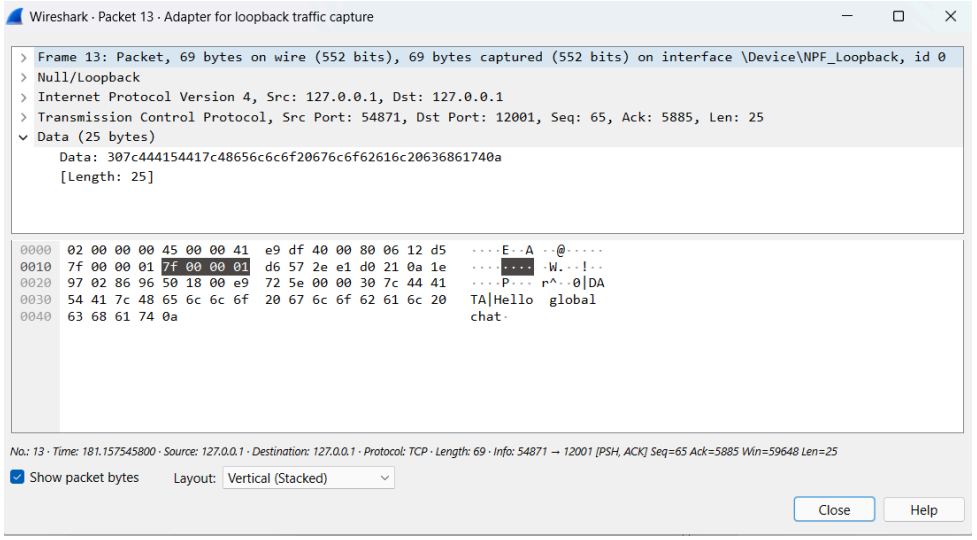


Figure 8.4 shows a packet containing a message sent on the global chat

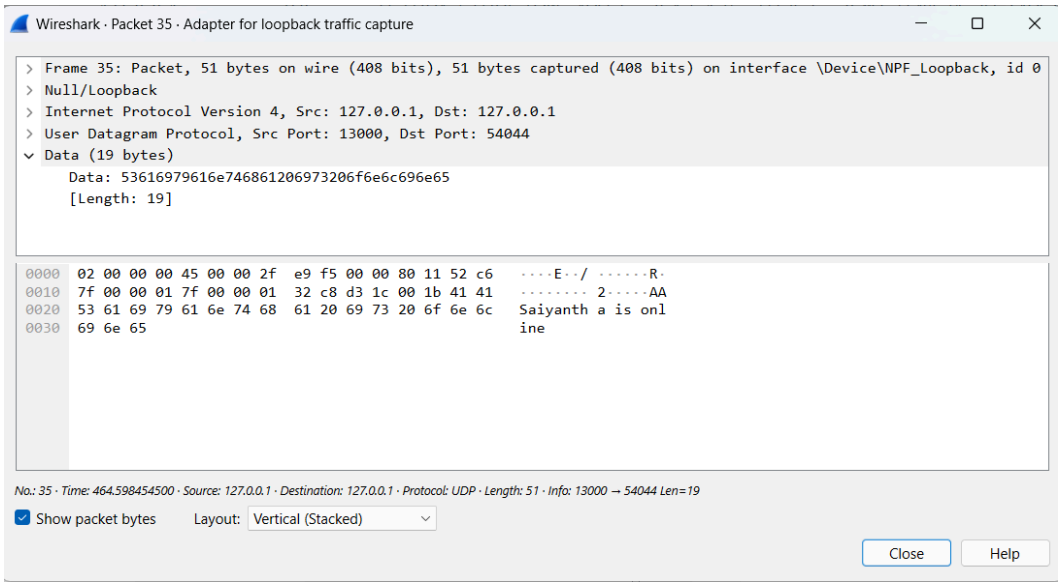


Figure 8.5 shows a UDP packet

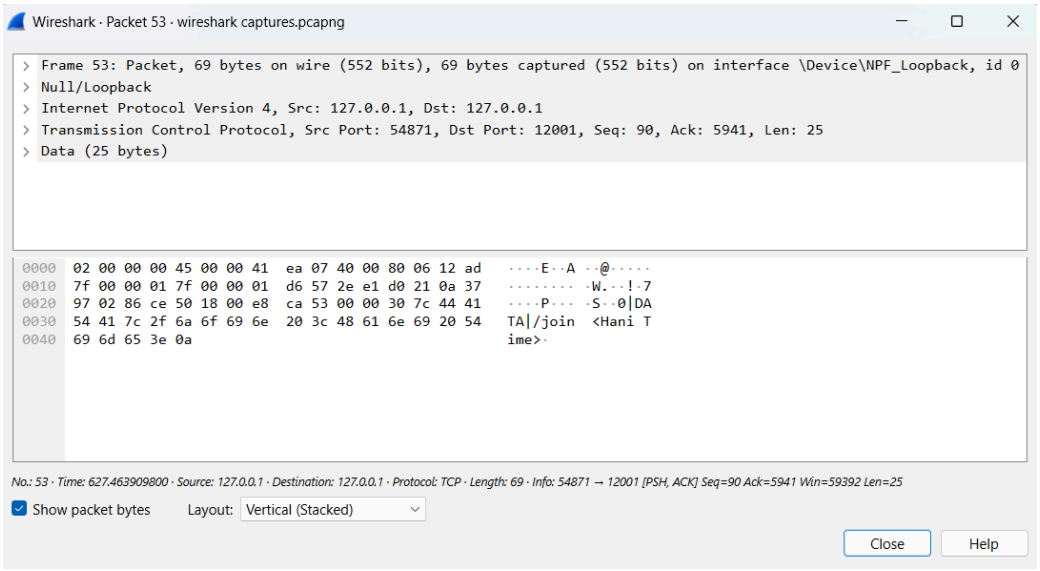
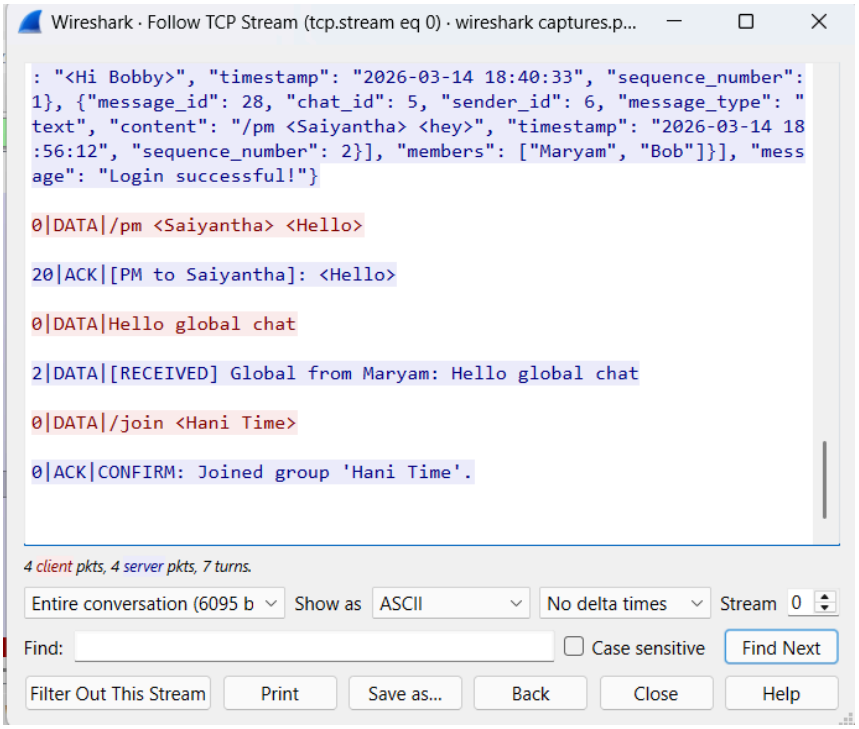


Figure 8.6 shows a packet sent to the server when a user creates a new group



1. Figure 9 shows a capture of the TCP stream sent between a client and server

Protocol Specification

Roles of Communicating Entities:

- **tcp_client.py:**
  This program acts as a client and peer of our application of which multiple instances can be opened at once. It is a peer since it allows users to send media/image, and document files directly to other peers by means of P2P communication and subverting server involvement. Every instance of a client has their own unique port which communicates with the server by maintaining a connection with it. The client is chiefly responsible for collecting user input and displaying output on the terminal for users to see and other than its peer to peer sharing capabilities all other actions are requested and fetched from the server.

- **tcp_server.py:**
  This program acts as the server of our application and together with **client_handler** this is the central hub of connection between clients and our database. It is here that the main TCP and UDP listening sockets are initialized as well as where all clients are associated with a specific thread.

- **client_handler.py:**
  This is the server's processing and logic central program and is mainly responsible for interpreting commands sent to the server by the client as well as almost all of our program's formatting. The client handler acts in tandem with *tcp_server.py* to form the server, helping also with database communication by enforcing user verification and authentication logic, as well as controlling message routing and most importantly message broadcasting.

- **protocol.py:**
  This code implements a simple application-layer protocol for client-server communication. Our encode_packet function is responsible for correctly formatting every outgoing message into a header with metadata and a body with the application data (seq|type|body) and converts it into bytes to be sent over our sockets. Our receive_packet function on the other hand reads in all data the socket until a newline to identify messages for parsing.

Communication Patterns:

The application works with a range of communication patterns creating a somewhat hybrid architecture. We have implemented both client-server TCP and UDP communication for general functionality including all message broadcasting and real time status updates respectively as well as Peer-to-peer TCP communication for file/media sharing. We chose this structure to capitalise off of TCPs reliability and accuracy, and UDPs speed.

Our client-side communication captures user input, and sends messages to the server via a TCP connection. It also listens for UDP online status broadcasts and hosts a local P2P socket for receiving direct file transfers. The server continuously listens to our designated TCP and UDP ports (12001, and 13000) and spawns independent connection threads every time a client joins.

Error Handling and Message Sequencing/Structure:

Together **client_handler** and **protocol.py** enforce global input, output and message structure, with client handler being mainly responsible for incorrect input handling, error validation and frontend-backend communication logic. **Client_handler** parses incoming requests from client terminals picked up by the server socket and acts as a logic core verifying user accessibility rights, database authentication, interpreting user requests from menus and commands. It controls how to broadcast all messages to other online clients or to a database store for those offline. **Protocol.py** mainly enforces the project strict Header:Body message requirements in our outline.

Implementation Challenges

In the beginning, the key to the assignment required combining different communication models in a single system. It was decided the application required a client-server communication for logging in, private messages, group messages, chat broadcasts and chat history. The system used peer-to-peer communication for file sharing. The system would need to operate successfully with the client, server and database.

The socket programming brought about some new issues, since TCP is stream based. File sharing requires the receiver to separate the file header from the file content correctly, this makes parsing more complicated. The server and client both needed threads to handle multiple actions without issue so the system required an understanding of concurrency.

Performance and Reliability Discussion

The system successfully works well under normal usage. TCP provides order and reliable delivery for authentication, private messaging, group messaging and file-transfer control. The server stays responsive even with multiple active users.

The peer-to-peer approach chosen benefits the file sharing since files are sent directly to clients instead through a server. This ensures the server isn't overloaded. Due to limitations of UDP, occasional packet loss does occur and disconnecting during file transfers does not guarantee the file will be transferred when connected again.

Basic Security Considerations

The immediate concern is that communication is not encrypted, messages and file transfers can be intercepted on the network. To help improve security, passwords are hashed before storage to ensure wrongful database access isn't as consequential

UDP does not provide built in authentication so it can be vulnerable to spoofing. File sharing does require validation but nothing high end that would counteract more complex attempts at malicious intent.

The system does not have much security in regards to repeated connection attempts and malformed requests. It currently has limited protection against denial-of-service style behaviour.

AI Declaration

AI was used to provide clarity and refine the written sections of this assignment. It was used to help with understanding of topics and with the structuring and layout of our reports.

AI was used for advising and improving on our system architecture and chat application design. In the coding portion of the assignment AI was used for determining necessary methods for the functionality of our chat application.

AI was definitely used for a significant amount of debugging, chiefly for quickly recognizing syntax errors and unexpected user input handling we'd otherwise not have accounted for/ thought of. AI was used a lot in advice for pushing git commits to our shared github repository to try our best to avoid future merge conflicts as well as help in resolving them when we inevitably faced some down the line.

Individual Contributions

The development of the chat application was done as a collective with each individual member taking responsibility for specific parts of the system:

Maryam was largely responsible for the implementation of the server component of the chat application. This involved creation of the TCP server responsible for handling client connections and routing of messages. She was also involved in creating parts of the database application that required interaction with the server.

Daahil was responsible for the creation and structure of the database. This involved creation of the database schema and tables used for storing user information. He also helped largely with the peer-to-peer file sharing feature in the client application.

Saiyantha was responsible for creating the client side of the application including the user interface and console command handling. Saiyantha also was responsible for the UDP status messages and parts of the peer-to-peer file sharing methods.

Collectively we assisted each other with resolving errors in the different sections of code. The design and system architecture was determined by all group members and all technical reports were created in collaboration. Changes to the system were all agreed upon by the team before implementation. Testing and debugging of the system was also done by all group members to ensure proper functionality of the entire application.

Git Log

Most Recent git log:

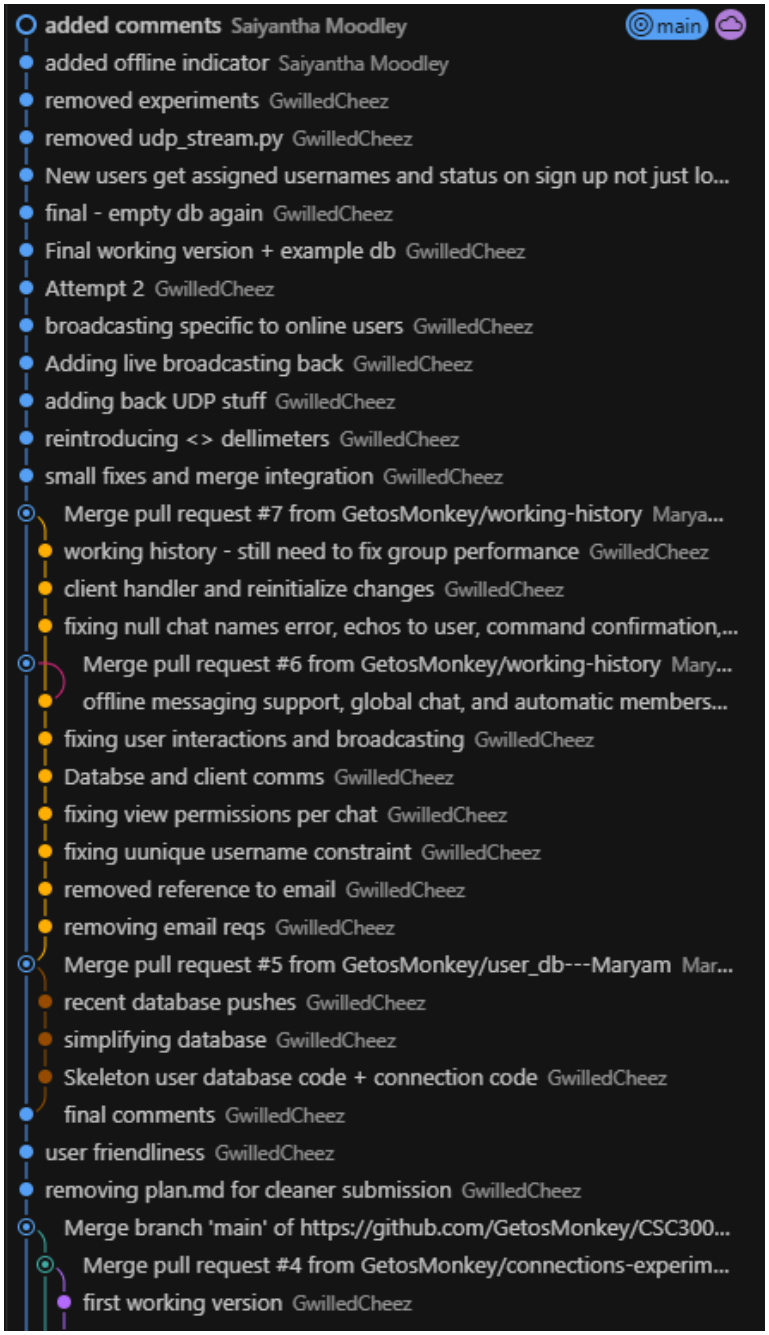
```
commit be181c9320e9c65184affde476128cbe7d26012a (HEAD -> main, origin/main,
origin/HEAD)
Author: Saiyantha Moodley <mdlsai010@myuct.ac.za>
Date:   Sun Mar 15 10:33:36 2026 +0200

    added comments

commit 55823773525fccad00d2dcf8e655c01e7cac816f
Author: Saiyantha Moodley <mdlsai010@myuct.ac.za>
commit be181c9320e9c65184affde476128cbe7d26012a (HEAD -> main, origin/main,
origin/HEAD)
Author: Saiyantha Moodley <mdlsai010@myuct.ac.za>
Date:   Sun Mar 15 10:33:36 2026 +0200

    added comments

commit 55823773525fccad00d2dcf8e655c01e7cac816f
Author: Saiyantha Moodley <mdlsai010@myuct.ac.za>
Date:   Sat Mar 14 19:35:15 2026 +0200

commit be181c9320e9c65184affde476128cbe7d26012a (HEAD -> main, origin/main,
origin/HEAD)
Author: Saiyantha Moodley <mdlsai010@myuct.ac.za>
Date:   Sun Mar 15 10:33:36 2026 +0200

    added comments

commit 55823773525fccad00d2dcf8e655c01e7cac816f
Author: Saiyantha Moodley <mdlsai010@myuct.ac.za>
commit be181c9320e9c65184affde476128cbe7d26012a (HEAD -> main, origin/main,
origin/HEAD)
Author: Saiyantha Moodley <mdlsai010@myuct.ac.za>
Date:   Sun Mar 15 10:33:36 2026 +0200

    added comments
commit be181c9320e9c65184affde476128cbe7d26012a (HEAD -> main, origin/main,
origin/HEAD)
Author: Saiyantha Moodley <mdlsai010@myuct.ac.za>
Date:   Sun Mar 15 10:33:36 2026 +0200

    added comments

commit 55823773525fccad00d2dcf8e655c01e7cac816f
commit be181c9320e9c65184affde476128cbe7d26012a (HEAD -> main, origin/main,
origin/HEAD)
Author: Saiyantha Moodley <mdlsai010@myuct.ac.za>
Date:   Sun Mar 15 10:33:36 2026 +0200
```

added comments

Github link:

https://github.com/GetosMonkey/CSC3002F-Networks-Assignment-Socket-Programming-Project.git

Git Commit Tree:



first working version GwilledCheez
adding logout deature GwilledCheez
Adding log statements to our code GwilledCheez
Working bare bones version GwilledCheez
added gitignore and database files GwilledCheez
gitignore file GwilledCheez
Latest working version of tcp_client.py - from Saiyantha GwilledC...
Merge pull request #3 from GetosMonkey/maryam-branch Geto S...
fixed authentication logic, hidden whitespace error, threading race ...
Merge pull request #2 from GetosMonkey/maryam-branch Geto ...
fixing server in use issues GwilledCheez
Fixing combination/merge conflicts between client and server -test...
Merge branch 'main' into maryam-branch GwilledCheez
Merge pull request #1 from GetosMonkey/client: Merge client br...
added menus Saiyantha Moodley
Changes made to start_client() method and threading added Sai...
Added user menu and started main method Saiyantha Moodley
test run 1 GwilledCheez
removed tutorial code GwilledCheez
simplifying from binary to text protocal GwilledCheez
Recently updated GwilledCheez
latest uupdate GwilledCheez
updated protocal GwilledCheez
Added files GwilledCheez